Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07 Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 4

Дисциплина МДК 03.01 Внедрение и поддержка компьютерных систем

Тема: Реализация версионирования API и проверка обратной совместимости в ASP.NET Core

Предметный домен: каталог интернет-магазина

Студент (ка)/учащийся: Иванов Егор Михайлович / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: Чудинова А.А. /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «08» октября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 СОЗДАНИЕ ПРОЕКТА И ПОДКЛЮЧЕНИЕ ПАКЕТОВ 4
2 НАСТРОЙКА PROGRAM.CS 5
3 РЕАЛИЗАЦИЯ КОНТРОЛЛЕРОВ V1 И V2 7
4 ТЕСТИРОВАНИЕ ЧЕРЕЗ SWAGGER UI 10
ВЫВОДЫ 13
ПРИЛОЖЕНИЕ А. ПРОЕКТ И ДОПОЛНИТЕЛЬНАЯ ПРОВЕРКА 14

ВВЕДЕНИЕ

Цель работы — закрепить на практике навыки проектирования параллельных версий REST API (v1 и v2) в экосистеме ASP.NET Core, освоить принцип аддитивных изменений (Additive Changes) и научиться защищать клиентские приложения от критических поломок (Breaking Changes).

В качестве предметного домена выбран каталог интернет-магазина. Разработан бэкенд на ASP.NET Core Web API, который одновременно обслуживает две версии эндпоинта списка товаров: версия 1 возвращает базовый набор полей, версия 2 — те же поля без изменений и дополнительные поля.

Для достижения цели решены следующие задачи:

- создан проект ApiVersioningLab.Ivanov по шаблону ASP.NET Core Web API;
- подключена библиотека версионирования Asp.Versioning.Mvc;
- в Program.cs настроены версионирование и генератор Swagger для версий v1 и v2;
- реализованы контроллеры ProductsV1Controller и ProductsV2Controller;
- выполнено тестирование обеих версий через Swagger UI.

Инструменты: Microsoft Visual Studio 2026, .NET 10.0 (требование задания — .NET 8.0 и выше), пакеты NuGet Asp.Versioning.Mvc 8.1.0, Asp.Versioning.Mvc.ApiExplorer 8.1.0 и Swashbuckle.AspNetCore 6.9.0.

1 СОЗДАНИЕ ПРОЕКТА И ПОДКЛЮЧЕНИЕ ПАКЕТОВ

В Visual Studio создан проект по шаблону ASP.NET Core Web API (C#) с именем ApiVersioningLab.Ivanov, платформа — .NET 10.0, генерация контроллеров включена.

Через консоль диспетчера пакетов (Tools > NuGet Package Manager > Package Manager Console) установлены библиотеки:

Листинг 1 – Команды установки пакетов NuGet

```
Install-Package Asp.Versioning.Mvc
Install-Package Asp.Versioning.Mvc.ApiExplorer
Install-Package Swashbuckle.AspNetCore
```

Пакет Asp.Versioning.Mvc отвечает за маршрутизацию запросов по версиям. Пакет Asp.Versioning.Mvc.ApiExplorer передаёт сведения о версиях генератору документации, чтобы Swagger построил отдельный документ для каждой версии. Пакет Swashbuckle.AspNetCore добавлен потому, что в шаблоне .NET 10 Swagger UI по умолчанию не подключается. Итоговый файл проекта приведён в листинге 2.

Листинг 2 – Файл проекта ApiVersioningLab.Ivanov.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>ApiVersioningLab.Ivanov</RootNamespace>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Asp.Versioning.Mvc" Version="8.1.0" />
    <PackageReference Include="Asp.Versioning.Mvc.ApiExplorer" Version="8.1.0" />
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.9.0" />
  </ItemGroup>

</Project>
```

Структура решения: папка Controllers содержит контроллеры двух версий, папка Models — контракты ответов (DTO) и тестовые данные, папка Swagger — класс настройки документации.

2 НАСТРОЙКА PROGRAM.CS

В Program.cs зарегистрирована служба версионирования API. Версия по умолчанию — 1.0. Номер версии читается из сегмента URL (api/v1/…, api/v2/…). Параметр ReportApiVersions добавляет в ответ заголовок api-supported-versions со списком поддерживаемых версий. Метод AddApiExplorer формирует группы v1 и v2, а Swagger UI получает по одному описанию на каждую группу. Код приведён в листинге 3.

Листинг 3 – Файл Program.cs

```
using Asp.Versioning;
using ApiVersioningLab.Ivanov.Swagger;


var builder = WebApplication.CreateBuilder(args);


builder.Services.AddControllers();


// 1. Регистрация службы версионирования API
builder.Services
    .AddApiVersioning(options =>
    {
        options.DefaultApiVersion = new ApiVersion(1, 0);          // версия по
умолчанию
        options.AssumeDefaultVersionWhenUnspecified = true;         // если клиент
не указал версию — v1
        options.ReportApiVersions = true;                           // заголовки
api-supported-versions
        options.ApiVersionReader = new UrlSegmentApiVersionReader(); // версия
читается из сегмента URL
    })
    .AddMvc()
    // 2. Интеграция с ApiExplorer: Swagger получает отдельную группу на каждую
версию
    .AddApiExplorer(options =>
    {
        options.GroupNameFormat = "'v'VVV";            // имена групп: v1, v2
        options.SubstituteApiVersionInUrl = true;      // {version} подставляется в
маршрут
    });


// 3. Генератор Swagger: документ для каждой версии формируется автоматически
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.ConfigureOptions<ConfigureSwaggerOptions>();


var app = builder.Build();


if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(options =>
    {
        // Выпадающий список версий в Swagger UI: v1 и v2
        foreach (var description in app.DescribeApiVersions())
        {
            options.SwaggerEndpoint(
```

```
                $"/swagger/{description.GroupName}/swagger.json",
                $"Каталог интернет-магазина API
{description.GroupName.ToUpperInvariant()}");
        }
    });
}

app.UseAuthorization();
app.MapControllers();

app.Run();
```

Класс ConfigureSwaggerOptions (листинг 4) перебирает обнаруженные версии API и создаёт для каждой Swagger-документ с названием и описанием. Поэтому новые версии появляются в Swagger UI автоматически, без ручного дописывания кода.

Листинг 4 – Файл Swagger/ConfigureSwaggerOptions.cs

```
using Asp.Versioning.ApiExplorer;
using Microsoft.Extensions.Options;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;

namespace ApiVersioningLab.Ivanov.Swagger;

/// <summary>
/// Создаёт отдельный Swagger-документ для каждой обнаруженной версии API.
/// </summary>
public class ConfigureSwaggerOptions : IConfigureOptions<SwaggerGenOptions>
{
    private readonly IApiVersionDescriptionProvider _provider;

    public ConfigureSwaggerOptions(IApiVersionDescriptionProvider provider) =>
_provider = provider;

    public void Configure(SwaggerGenOptions options)
    {
        foreach (var description in _provider.ApiVersionDescriptions)
        {
            options.SwaggerDoc(description.GroupName, new OpenApiInfo
            {
                Title = "Каталог интернет-магазина API",
                Version = description.ApiVersion.ToString(),
                Description = description.ApiVersion.MajorVersion == 1
                    ? "Версия 1: базовый набор полей товара (id, title, price)."
                    : "Версия 2: поля v1 + аддитивные поля (currency, inStock,
category, apiVersionInfo)."
            });
        }
    }
}
```

3 РЕАЛИЗАЦИЯ КОНТРОЛЛЕРОВ V1 И V2

Для каждой версии описан собственный контракт ответа (листинг 5). В ProductV2Dto поля Id, Title и Price сохранены с теми же именами, типами и смыслом, что и в ProductV1Dto. Новые поля Currency, InStock, Category и ApiVersionInfo только добавлены.

Листинг 5 – Контракты ответов ProductV1Dto и ProductV2Dto

```
namespace ApiVersioningLab.Ivanov.Models;


/// <summary>Контракт товара версии 1 — базовый набор полей.</summary>
public record ProductV1Dto(int Id, string Title, decimal Price);


/// <summary>
/// Контракт товара версии 2. Поля v1 сохранены без изменений (имя, тип, смысл),
/// новые поля только добавлены — аддитивное изменение.
/// </summary>
public record ProductV2Dto(
    int Id,
    string Title,
    decimal Price,
    string Currency,
    bool InStock,
    string Category,
    string ApiVersionInfo);
```

Контроллер версии 1 (листинг 6) отмечен атрибутами [ApiVersion("1.0")] и [Route("api/v{version:apiVersion}/products")]. Он возвращает список товаров с полями id, title, price и статусом 200 OK, а также товар по идентификатору.

Листинг 6 – Файл Controllers/ProductsV1Controller.cs

```
using Asp.Versioning;
using ApiVersioningLab.Ivanov.Models;
using Microsoft.AspNetCore.Mvc;


namespace ApiVersioningLab.Ivanov.Controllers;


/// <summary>Каталог товаров — версия API 1.0.</summary>
[ApiController]
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV1Controller : ControllerBase
{
    /// <summary>Список товаров (базовый набор полей).</summary>
    [HttpGet]
    [ProducesResponseType(typeof(IEnumerable<ProductV1Dto>),
StatusCodes.Status200OK)]
    public IActionResult GetAll()
    {
```

```
        var result = ProductStore.Items.Select(p => new ProductV1Dto(p.Id, p.Title,
p.Price));
        return Ok(result);
    }

    /// <summary>Товар по идентификатору.</summary>
    [HttpGet("{id:int}")]
    [ProducesResponseType(typeof(ProductV1Dto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public IActionResult GetById(int id)
    {
        var p = ProductStore.Items.FirstOrDefault(x => x.Id == id);
        return p is null ? NotFound() : Ok(new ProductV1Dto(p.Id, p.Title,
p.Price));
    }
}
```

Контроллер версии 2 (листинг 7) использует тот же шаблон маршрута, но с атрибутом [ApiVersion("2.0")]. Он берёт данные из того же источника и возвращает расширенный контракт: старые поля плюс валюту, признак наличия на складе, категорию и описание версии.

Листинг 7 – Файл Controllers/ProductsV2Controller.cs

```
using Asp.Versioning;
using ApiVersioningLab.Ivanov.Models;
using Microsoft.AspNetCore.Mvc;

namespace ApiVersioningLab.Ivanov.Controllers;

/// <summary>Каталог товаров — версия API 2.0 (расширяет v1 аддитивными
полями).</summary>
[ApiController]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV2Controller : ControllerBase
{
    private const string VersionInfo =
        "v2.0: добавлены поля currency, inStock, category; поля v1 не изменены";

    /// <summary>Список товаров (поля v1 + новые поля).</summary>
    [HttpGet]
    [ProducesResponseType(typeof(IEnumerable<ProductV2Dto>),
StatusCodes.Status200OK)]
    public IActionResult GetAll()
    {
        var result = ProductStore.Items.Select(Map);
        return Ok(result);
    }

    /// <summary>Товар по идентификатору.</summary>
    [HttpGet("{id:int}")]
    [ProducesResponseType(typeof(ProductV2Dto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public IActionResult GetById(int id)
    {
        var p = ProductStore.Items.FirstOrDefault(x => x.Id == id);
        return p is null ? NotFound() : Ok(Map(p));
    }
```

```
    private static ProductV2Dto Map(Product p) =>
        new(p.Id, p.Title, p.Price, p.Currency, p.Stock > 0, p.Category,
VersionInfo);
}
```

4 ТЕСТИРОВАНИЕ ЧЕРЕЗ SWAGGER UI

Приложение запущено (F5), Swagger UI открыт по адресу http://localhost:5180/swagger. В списке Select a definition отображаются два документа: «Каталог интернет-магазина API V1» и «Каталог интернет-магазина API V2». Документ версии 1 с выполненным запросом GET /api/v1/products показан на рисунке 1.

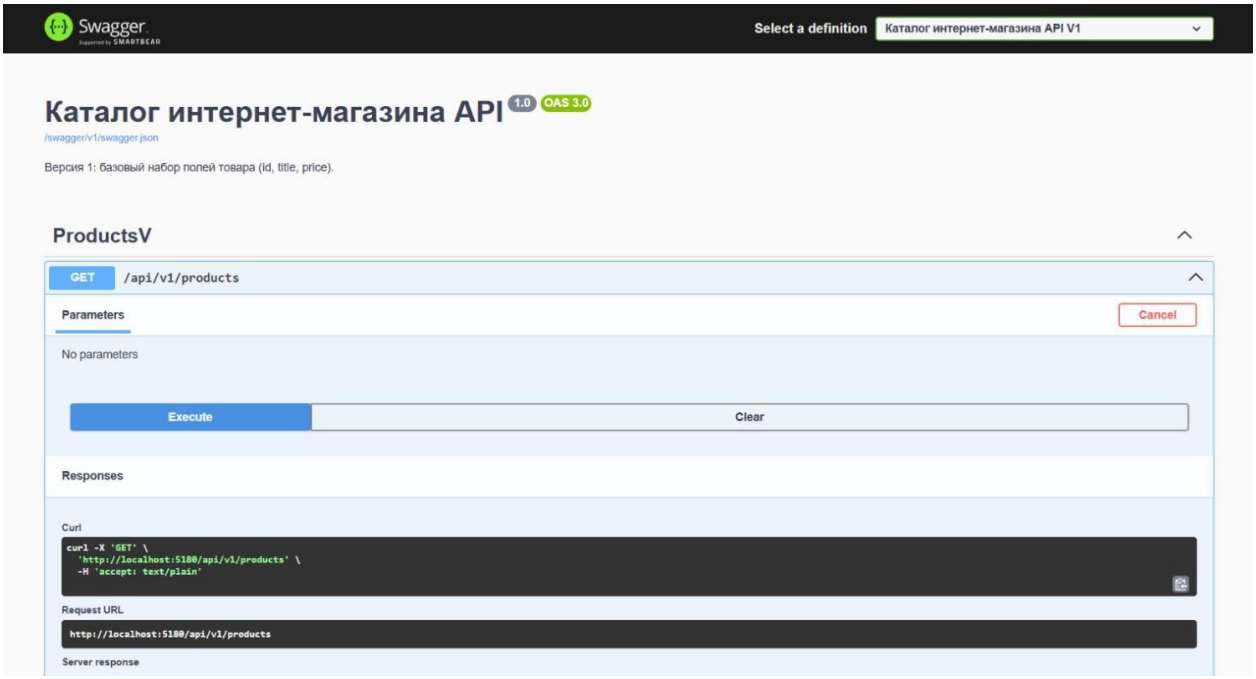


Рисунок 1 – Swagger UI, документ версии 1, запрос GET /api/v1/products

Ответ сервера для версии 1 приведён на рисунке 2. Код ответа — 200 OK, каждый товар содержит три поля: id, title, price. В заголовке api-supported-versions: 1.0, 2.0 сервер сообщает клиенту, какие версии он поддерживает.

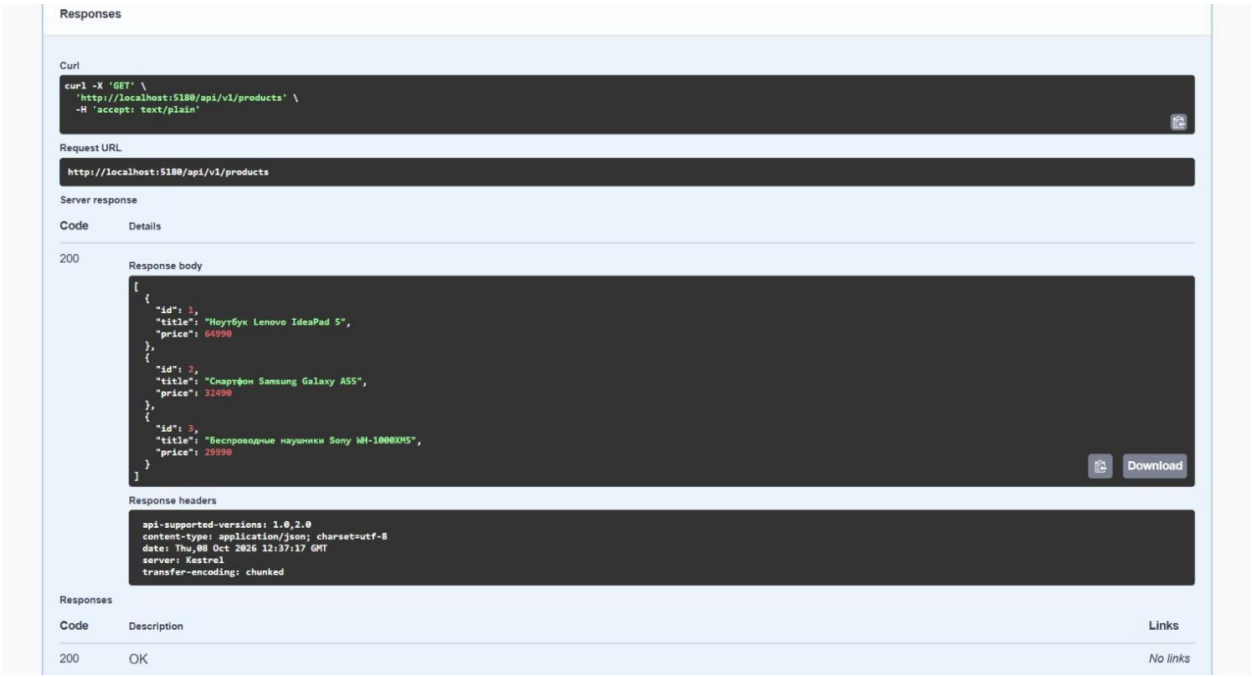


Рисунок 2 – Ответ сервера на запрос GET /api/v1/products

После переключения на документ V2 в Swagger UI отображается эндпоинт GET /api/v2/products (рисунок 3).

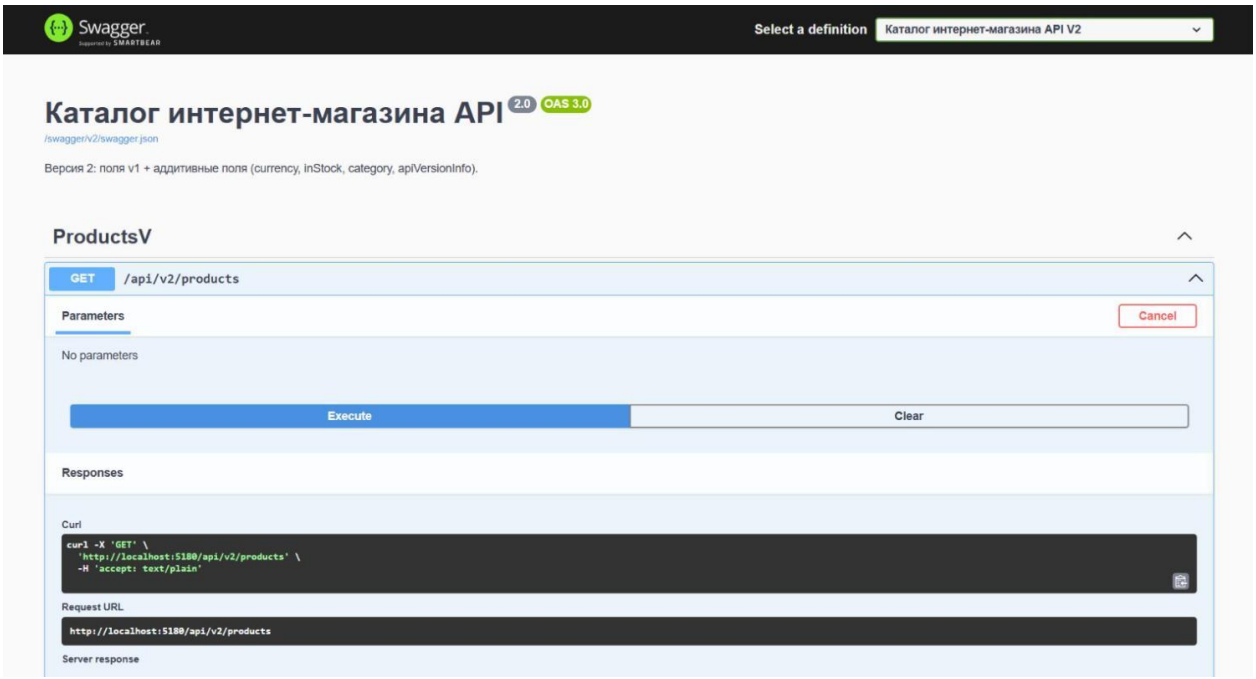


Рисунок 3 – Swagger UI, документ версии 2, запрос GET /api/v2/products

Ответ версии 2 (рисунок 4) также имеет код 200 OK. Поля id, title и price совпадают с версией 1 по именам, типам и значениям, а к ним добавлены поля currency, inStock, category и apiVersionInfo.

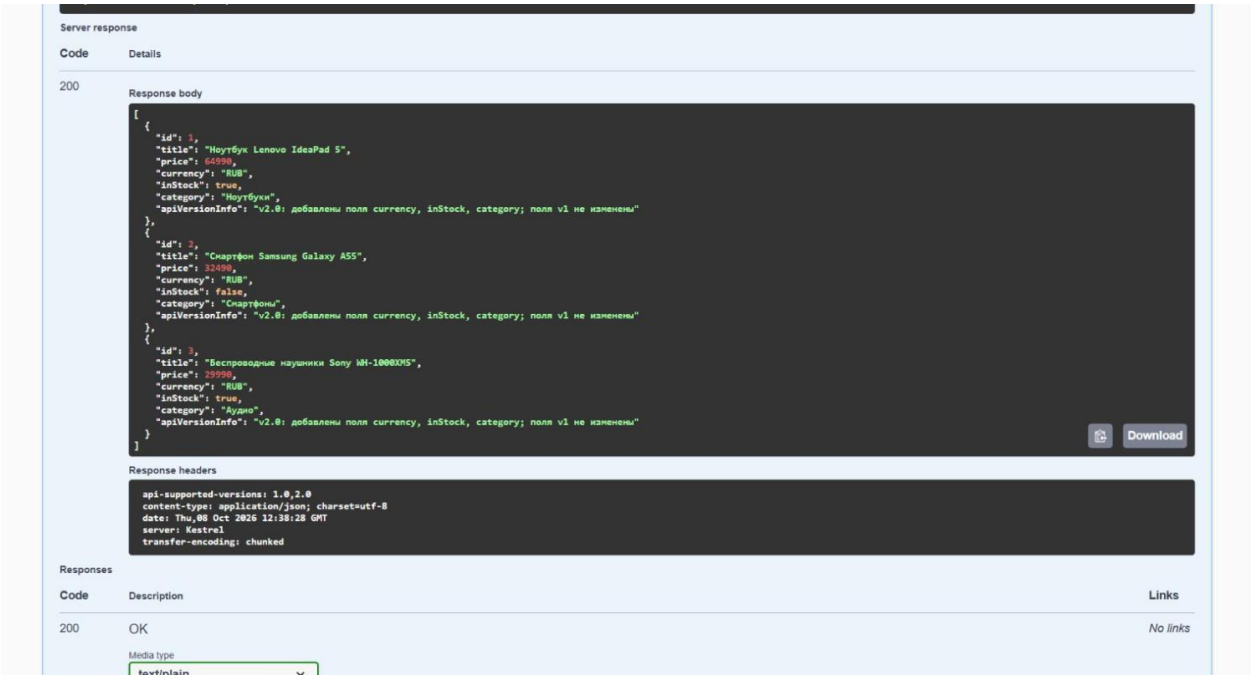


Рисунок 4 – Ответ сервера на запрос GET /api/v2/products

Сравнение полей ответов двух версий приведено в таблице 1.

Таблица 1 – Сравнение полей ответа версий v1 и v2

Поле	Тип	v1	v2	Характер изменения
id	int	есть	есть	не изменено
title	string	есть	есть	не изменено
price	decimal	есть	есть	не изменено
currency	string	—	есть	добавлено (аддитивно)
inStock	bool	—	есть	добавлено (аддитивно)
category	string	—	есть	добавлено (аддитивно)
apiVersionInfo	string	—	есть	добавлено (аддитивно)

Дополнительно проверены запросы к товару по идентификатору. GET /api/v1/products/2 вернул {"id":2,"title":"Смартфон Samsung Galaxy A55","price":32490}. GET /api/v2/products/2 вернул те же три поля с теми же значениями и четыре новых поля. Обе версии работают одновременно, и ответ v1 после появления v2 не изменился.

ВЫВОДЫ

В работе создано веб-приложение ASP.NET Core Web API, в котором параллельно работают версии v1 и v2 эндпоинта каталога товаров. Обе версии корректно отображаются в Swagger UI и возвращают 200 OK.

Добавление новых полей в версию v2 без изменения старых структур защищает клиентов от breaking changes. Поля id, title и price сохранили имена, типы и смысл, поэтому существующие клиенты продолжают получать и разбирать знакомые данные. Клиент, который не знает о новых полях, просто их игнорирует, а клиенты v1 по-прежнему обращаются к /api/v1 и получают прежний формат. Переход на v2 происходит постепенно, по готовности каждого клиента, а заголовок api-supported-versions показывает, какие версии доступны. Breaking change возникло бы при удалении, переименовании или смене типа поля, и такие изменения допустимы только в новой мажорной версии при сохранении старой.

ПРИЛОЖЕНИЕ А. ПРОЕКТ И ДОПОЛНИТЕЛЬНАЯ ПРОВЕРКА

Исходный код, архив проекта и отчёт опубликованы в репозитории GitHub: https://github.com/Egorvan606-senior/mdk0301-pr4-api-versioning

Код проекта восстановлен по листингам 2–7 этого отчёта. В исходном тексте не было листинга учебного источника товаров ProductStore, от которого зависят оба контроллера. Три товара и их цены восстановлены по снимкам ответов Swagger UI на рисунках 2 и 4; численные остатки на складе являются учебными и сохраняют показанные признаки inStock. Полный код приведён в листинге 8.

Листинг 8 – Файл Models/ProductStore.cs

```
namespace ApiVersioningLab.Ivanov.Models;
/// <summary>Учебные данные каталога, восстановленные по результатам Swagger в
отчёте.</summary>
public record Product(int Id, string Title, decimal Price, string Currency, int Stock, string
Category);
public static class ProductStore
{
    public static IReadOnlyList<Product> Items { get; } = new[]
    {
        new Product(1, "Ноутбук Lenovo IdeaPad 5", 64990m, "RUB", 10, "Ноутбуки"),
        new Product(2, "Смартфон Samsung Galaxy A55", 32490m, "RUB", 0, "Смартфоны"),
        new Product(3, "Беспроводные наушники Sony WH-1000XM5", 29990m, "RUB", 5, "Аудио")
    };
}
```

Проект собран в Visual Studio без ошибок и предупреждений (рисунок 5). Для повторного запуска в архив включены решение ApiVersioningLab.Ivanov.slnx и файл запросов ApiVersioningLab.Ivanov.http. Скрипт verify_api.py выполнил семь проверок по реальным HTTP-запросам: ответы v1 и v2 для трёх товаров, сохранение полей v1, новые поля v2, поиск по id, код 404 и два документа Swagger. Все семь проверок завершились успешно. Результаты приведены в таблице 2.

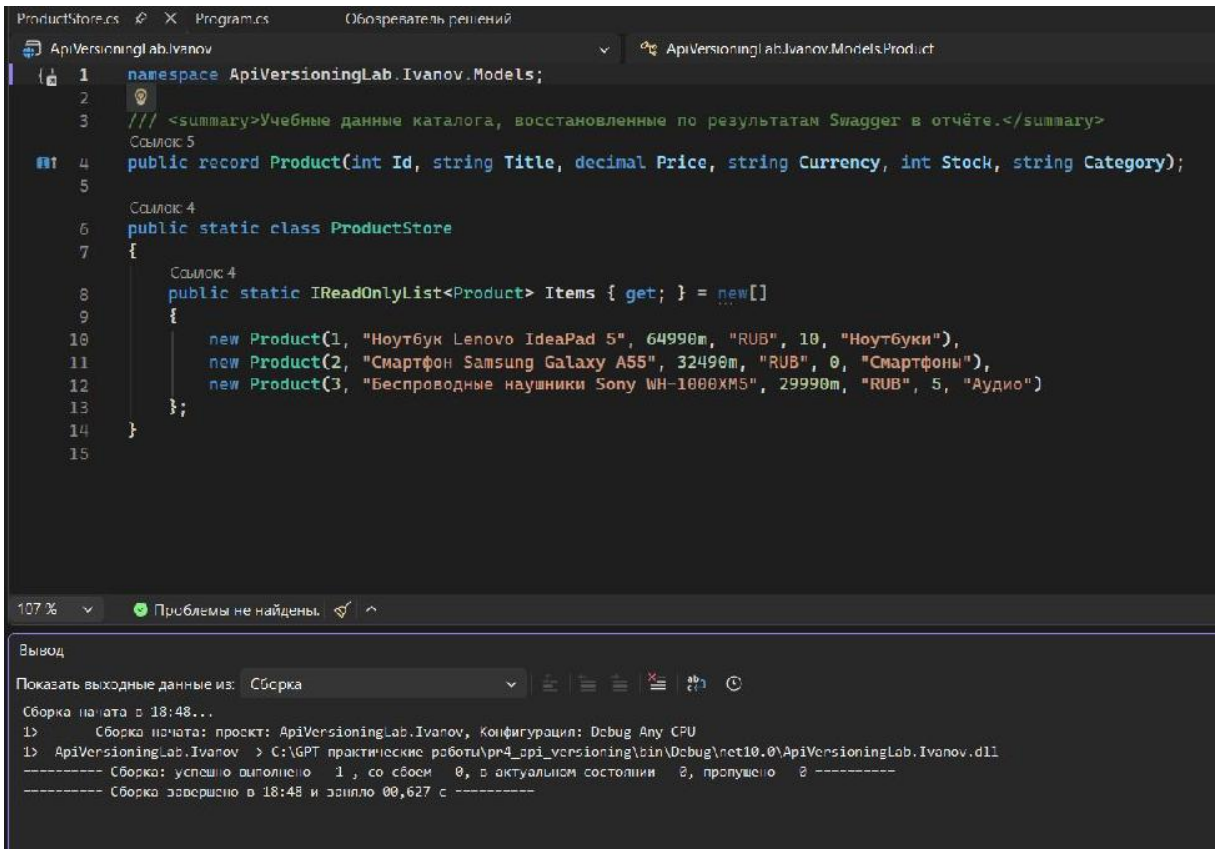


Рисунок 5 – Проект ПР4 и успешная сборка в Visual Studio

Таблица 2 – Результаты дополнительных проверок API

Проверка	Результат
Ответы v1 и v2	HTTP 200; три товара
Контракт полей	v1: 3 поля; v2: 7 полей
Обратная совместимость	Поля и значения v1 сохранены
Заголовки версий	1.0 и 2.0 указаны
Поиск товара по id	Обе версии: HTTP 200
Несуществующий id	Обе версии: HTTP 404
Документы Swagger	v1 и v2 доступны